AMRITA VISHWA VIDYAPEETHAM

AHEAD Online

Data Structures and Algorithms

Geetha Lekshmy V
Department of Computer Science
Amrita Vishwa Vidyapeetham

Objectives

Algorithm Design Techniques

Place your
Webcam Video here
Size 100%

Algorithm Design Techniques

Brute Force

Divide and Conquer

Dynamic Programming

Greedy Strategy

Backtracking

Branch and Bound

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Algorithm Design Techniques

Brute Force

- Try out every possibility

- Example
  - Linear Search – Brute Force method
  - Binary Search – Not a Brute Force Method

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Algorithm Design Techniques

- Divide and conquer

✓ Divide the original problem into smaller instances of the same problems.

✓ Smaller instances are solved(typically recursively).

✓ Combine the solutions to the smaller instances to form the solutions for the original instance.

Typically the running time of a divide and conquer algorithm is

$T(n)=aT(n/b) + f(n)$

n→ input size

a→ no. of sub problems

n/b→size of a sub problem

f(n)→ time for dividing and combining the solution.

Algorithm Design Techniques

- Dynamic Programming

✓ Overlapping sub-problems (Sub-problems overlap when they share subsub problems).

Solve smaller sub-problem once and store the results in a table, avoiding recomputing every time a sub problem is solved.

✓ Optimal Substructure

If an optimal solution contain optimal subsolution the problem then it exhibits optimal sub structure.

Algorithm Design Techniques

Greedy Approach

- ✓ Apply to optimization problems(Optimization problems refers to the class of problems where the goal is to maximize or minimize the solution.) in which we make a set of choices to arrive at an optimal solution.
- ✓ A greedy algorithm works if a problem exhibits two properties
  - Greedy Choice Property – A globally optimal solution can be arrived at by making locally optimal solutions. An optimal solution can be obtained by making greedy choices(make a choice that looks best at that moment).
  - Optimal Substructure – Optimal solution contain optimal sub solutions. Solutions to sub problems of an optimal solution are optimal.

Algorithm Design Techniques

Backtracking

- Applied to problems with multiple solutions, not optimization problems.
- Systematic brute force approach.

- Construct candidate solutions one component at a time and evaluate partially constructed solutions, if it is not leading to a solution, remaining components are not generated.

- State Space Tree is constructed whose node reflect the choices made for solution components.

Algorithm Design Techniques

Branch and Bound

- Applicable for optimization problem.
- Typically exponential in terms of time complexity.
- Generates state space Tree.
- Every node in the state space has a bound on the best value of the objective fn on any soln.
- Search path at the current node is terminated if the value of node's bound is not better than the value of best soln.

Summary

Different algorithm design techniques.

Place your
Webcam Video here
Size 100%

Objectives

Quick Sort

Example



Quick Sort

- Uses Divide and Conquer paradigm.

- Divide and conquer - an algorithm design technique
  - ✓ Divide the original problem into subproblems.
  - ✓ Solve every subproblem individually.
  - ✓ Combine the solutions to the subproblems to form the solutions for the original problem.

Quick Sort

Given an array A of size n, say A[l..n]

- ✓ Divide the array into two sub arrays A[l..m-1] and A[m+1..n], such that elements in A[l..m-1] is less than or equal to A[m], which is in turn less than or equal to A[m+1..n].
- ✓ Sort the subarrays A[l..m-1] and A[m+1..n] recursively.

Quick Sort

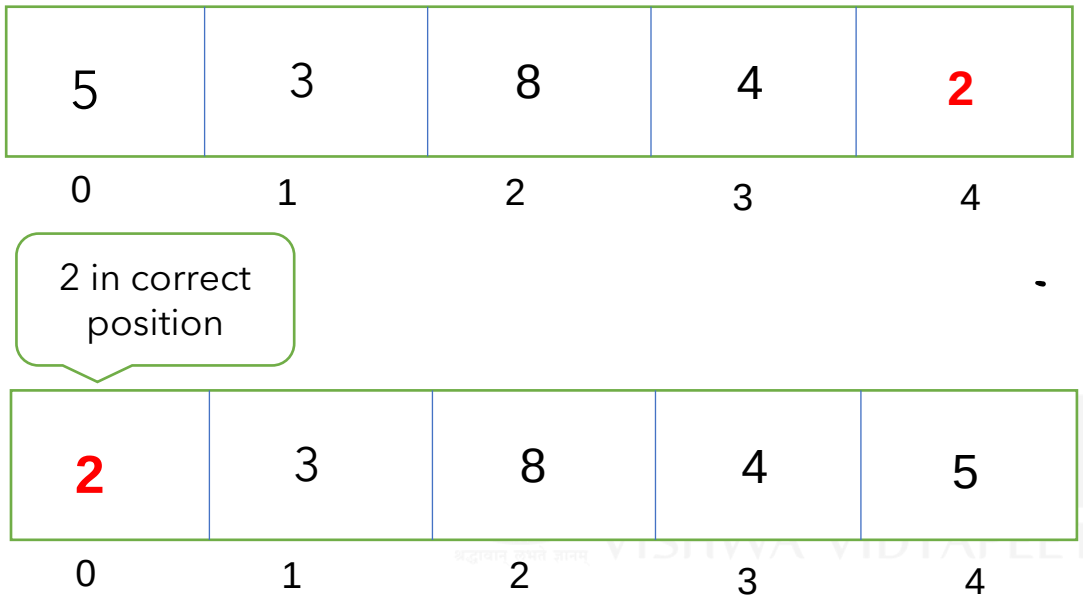
```
1 QuickSort(A,l,n):
2 |   if l < n then
3 |   |   m =Partition(A,l,n)
4 |   |   QuickSort(A,l,m − 1)
5 |   |   QuickSort(A,m + 1,n)
6 |   end
7 end
```

Quick Sort - Partition

```
1  Partition(A, l,n):
2      pivot = A[n]
3      i = l − 1
4      j = l
5      while j ≤ n − 1 do
6          if A[j] ≤ pivot then
7              i = i + 1
8              swapA[i], A[j]
9          end
10         j = j + 1
11     end
12     swap(A[i + 1], A[n]
13     return i + 1;
14 end
```

- pivot=A[n] – selects the last element as pivot.
- In the given loop pivot is compared with each element in the array.
- If the element in the array A[j] is less than pivot, exchange it with an element at i+1th position, which to the left of A[j] and greater than pivot.
- Variable **i** keeps track of positions to store elements less than pivot.
- After examining all the elements in the array in the loop, finally the pivot is placed at i+1 th position.
- Partition returns the position of pivot.
- After partitioning, pivot will be at the correct position in the array. All elements to the left of pivot will be smaller and elements to the right of pivot will be larger.

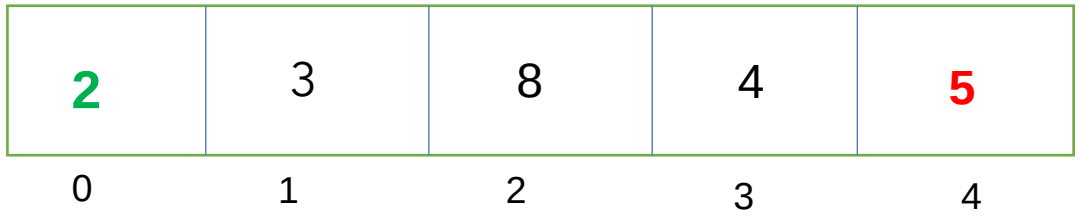
Quick Sort – Example




```
QuickSort(A,0,4)
    if 0 < 4
      m=Partition(A,0,4)
       QuickSort(A,0,m-1)
        QuickSort(A,m+1,4)
```

```
Partition(A,0,4)
   pivot = A[4] =2
    i=0-1 =-1
     j=0
     while(j<3)
        if A[j] < 2
          i=i+1
          swap (A[i],A[j])
       j=j+1
     swap(A[i+1], A[n])
   return i+1
```

Quick Sort – Example








```
QuickSort(A,0,4)
    if 0 < 4
      0
     m=Partition(A,0,4)
                    -1
     QuickSort(A,0,m-1)
                    1
     QuickSort(A,m+1,4)
```

terminates

```
QuickSort(A,1,4)
    if 1 < 4

      m=Partition(A,1,4)

      QuickSort(A,1,m-1)

      QuickSort(A,m+1,4)
```

Quick Sort - Example

Trace quick sort algorithm with the following input

10	23	18	2	17
0	1	2	3	4

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Summary

Quick Sort

What if Quick Sort takes a set of sorted numbers as input?

Place your
Webcam Video here
Size 100%

Reference

Introduction to Algorithms – Thomas H Cormen

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Data Structures and Algorithms

Aswathy R Nair
Geetha Lekshmy V
Department of Computer Science
Amrita Vishwa Vidyapeetham

Objectives

Quicksort - Analysis

Place your
Webcam Video here
Size 100%

Divide and Conquer - Analysis

T(n) → time taken to solve the problem of size 'n'

If problem is divided into *a* sub problems, each of size n/b, then
T(n) = aT(n/b) + f(n)

T(n/b) →time taken to solve a sub-problem of size n/b.

F(n)→ total work done in each recursion, which is sum of time taken to divide the problem into sub-problems and time taken to combine the solutions of sub problems.

$$T(n)=\begin{cases} \theta(1) \quad for\ base\ case, \\ aT(n/b) + f(n) \text{ otherwise.} \end{cases}$$

Quicksort

```
1  QuickSort(A,l,n):
2      if l < n then
3          m =Partition(A,l,n)
4          QuickSort(A,l,m − 1)
5          QuickSort(A,m + 1,n)
6      end
7  end
```

```
1  Partition(A, l,n):
2      pivot = A[n]
3      i = l − 1
4      j = l
5      while j ≤ n − 1 do
6          if A[j] ≤ pivot then
7              i = i + 1
8              swapA[i], A[j]
9          end
10         j = j + 1
11     end
12     swap(A[i + 1], A[n]
13     return i + 1;
14 end
```

Time complexity depends on **how the array is partitioned**.

- If array is divided into two halves then split is balanced and it is best case.
- If array is partitioned into one subarray with n-1 elements and another with 0 elements, this is worst case.

$$T(n)=\begin{cases} \theta(1) \quad for\ base\ case, \\ T(k-1)+T(n-k)+f(n) \text{ otherwise.} \end{cases}$$

Here k is the position of pivot, or position where array is split.

Quicksort – Best case Analysis

```
1  QuickSort(A,l,n):
2      if l < n then
3          m =Partition(A,l,n)
4          QuickSort(A,l,m − 1)
5          QuickSort(A,m + 1,n)
6      end
7  end
```

```
1  Partition(A, l,n):
2      pivot = A[n]
3      i = l − 1
4      j = l
5      while j ≤ n − 1 do
6          if A[j] ≤ pivot then
7              i = i + 1
8              swapA[i], A[j]
9          end
10         j = j + 1
11     end
12     swap(A[i + 1], A[n]
13     return i + 1;
14 end
```

Best Case Partitioning – When the array is partitioned in the middle.

Pivot placed in the middle

With 'n' elements, when the array is partitioned into two, each of the sub array will contain no more than n/2 elements.

Size of one sub array will be $\lfloor n/2 \rfloor$ and other will be $\left\lceil \frac{n}{2} \right\rceil - 1$.

$$T(n) = \begin{cases} \theta(1) & if\ n \leq 1, \\ T(n/2) + T(n/2) + \theta(n) & \text{otherwise.} \end{cases}$$

Solving the recurrence, **T(n) = θ(nlogn).**



Quick Sort –Worst Case Analysis

Worst Case Partitioning – When the array of size 'n' is partitioned into one subarray with n-1 elements and another with 0 elements.

The pivot is positioned at first/last position of the array, in each recursive call ie k=1
Then the recurrence will be, $T(n) = T(n-1) + T(0) + \theta(n)$

Base case is reached after n recursions. So $T(n) = n * \theta(n) =$ **$\theta(n^2)$.**

Example of worst-case partitioning:- When **sorted set** of numbers is given as input to quick sort.

Summary

Best case and Worst-case Analysis of Quick sort



References

- Introduction to Algorithms - Cormen

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online